EXPERIMENT 11

Write the Python program for Map Coloring to implement the A* algorithm

Register Number: **192511013**

192511013

Aim

To write a Python program that colours the regions of a map (Australia) with three colours so that no two adjacent regions have the same colour, by formulating map colouring as a search problem and solving it with the A* algorithm.

Algorithm

Step 1: Represent the map as a dictionary of regions and their neighbours, and choose the set of available colours (Red, Green, Blue).

Step 2: Define a state as the list of colours assigned to the first k regions (in a fixed order). The start state is empty and the goal state has all regions coloured.

Step 3: Define the cost g(n) as the number of regions coloured so far and the heuristic h(n) as the number of regions still uncoloured. Then f(n) = g(n) + h(n). Since h(n) never over-estimates the remaining work, A* is optimal.

Step 4: Put the start state in a priority queue ordered by f(n); ties are broken in favour of the state with the smaller h(n).

Step 5: Remove the state with the lowest f(n). If all regions are coloured, it is the solution; stop.

Step 6: Otherwise take the next uncoloured region and try each colour. A colour is allowed only if no neighbour of that region already has it (constraint check). Push each allowed new state into the queue with its f value.

Step 7: Repeat from step 5 until a goal state is found or the queue is empty. Print the colour of every region, verify that adjacent regions differ, and print the number of states expanded.

192511013

Program

```
import heapq
from itertools import count

# Map of Australia: region -> neighbouring regions
regions = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
neighbours = {
    'WA':  ['NT', 'SA'],
    'NT':  ['WA', 'SA', 'Q'],
    'SA':  ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q':   ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V':   ['SA', 'NSW'],
    'T':   [],
}
colors = ['Red', 'Green', 'Blue']

def is_valid(assignment, region, color):
    return all(assignment.get(n) != color for n in neighbours[region])

def a_star_map_coloring():
    # state = tuple of colours given to regions[0..k-1]
    # g(n) = regions coloured so far, h(n) = regions still uncoloured
    # f(n) = g(n) + h(n); h never over-estimates, so A* is optimal
    tie = count()
    start = ()
    open_list = [(len(regions), len(regions), next(tie), start)]   # (f, h, tie, state)
    expanded = 0
    while open_list:
        f, h, _, state = heapq.heappop(open_list)
        expanded += 1
        if len(state) == len(regions):
            return dict(zip(regions, state)), expanded
        region = regions[len(state)]
        assignment = dict(zip(regions, state))
        for color in colors:
            if is_valid(assignment, region, color):
                new_state = state + (color,)
                g = len(new_state)
                h_new = len(regions) - g
                heapq.heappush(open_list, (g + h_new, h_new, next(tie), new_state))
    return None, expanded

if __name__ == "__main__":
    print("Colours available:", ', '.join(colors), "\n")
    print("Neighbours:")
    for r in regions:
        print(f"  {r:<4}: {neighbours[r]}")
    solution, expanded = a_star_map_coloring()
    if solution:
        print("\nColouring found by A* search:")
        for r in regions:
            print(f"  {r:<4}-> {solution[r]}")
        ok = all(solution[r] != solution[n] for r in regions for n in neighbours[r])
        print("\nNo two adjacent regions share a colour:", ok)
        print("States expanded:", expanded)
    else:
        print("No valid colouring exists")
```

192511013

Output

```
Colours available: Red, Green, Blue

Neighbours:
  WA  : ['NT', 'SA']
  NT  : ['WA', 'SA', 'Q']
  SA  : ['WA', 'NT', 'Q', 'NSW', 'V']
  Q   : ['NT', 'SA', 'NSW']
  NSW : ['Q', 'SA', 'V']
  V   : ['SA', 'NSW']
  T   : []

Colouring found by A* search:
  WA  -> Red
  NT  -> Green
  SA  -> Blue
  Q   -> Red
  NSW -> Green
  V   -> Red
  T   -> Red

No two adjacent regions share a colour: True
States expanded: 8
```

192511013